

7강

리다이렉트 & thymeleaf



리다이렉트(Redirect)





spring

➤ 리다이렉트(redirect) 란 ?

- 리다이렉트는 웹 브라우저(클라이언트)가 어떤 URL로 요청을 보냈을 때, 서버가 그 주소 말고 이 주소로 다시 가세요! 라고 명령하는 것을 말한다.
- redirect는 Model 데이터를 유지하지 않는다.
- redirect는 **새로운 HTTP 요청한다.**
- request / model 전부 초기화한다.

 일상생활 비유

여러분이 맛집 A를 찾아갔는데, 가게 문에 저희 옆 동네 B로 확장이전했습니다. 그리로 오세요!라는 안내문이 붙어 있는 것과 같습니다. 여러분은 그 안내를 보고 직접 B라는 새로운 주소로 발걸음을 옮기게 되죠.



spring

➤ 리다이렉트(redirect) 작동 원리

1. **요청 (Request):** 사용자가 브라우저에 `/old-path`를 입력합니다.
2. **응답 (Response):** 서버는 HTTP 상태 코드 **302** (또는 301)와 함께 `Location: /new-path`라는 정보를 담아 응답한다.
3. **재요청 (Re-request):** 브라우저는 이 응답을 받자마자, 사용자가 시키지 않아도 자동으로 `/new-path` 주소로 다시 요청을 보낸다.
4. **결과적으로** 사용자의 브라우저 주소창은 `/new-path`로 바뀌게 된다.

➤ 리다이렉트(redirect) 사용하는 이유

- **로그인 성공 후:** 로그인 페이지에 계속 머물면 안 되므로 메인 페이지로 보낼 때.
- **글쓰기 완료 후:** 게시글 작성을 마치고 "새로고침"을 눌렀을 때 글이 중복으로 써지는 것을 방지하기 위해 목록 페이지로 보낼 때 (**Post-Redirect-Get 패턴**).
- **페이지 주소가 바뀌었을 때:** 이전 주소로 들어온 사용자를 새 주소로 안내할 때.



spring

➤ 리다이렉트(redirect) 사용하기

- sample_redirect 프로젝트 생성 -> src/main/java -> MyController.java 생성
- 스프링 부트 컨트롤러에서 리다이렉트를 구현하는 방법은 아주 간단하다. 반환하는 문자열 앞에 **redirect:** 접두사를 붙여 준다.

```
package com.example.demo;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MyController {

    @GetMapping("/old-url")
    public String oldPage() {
        // "/new-url"로 리다이렉트 하라는 명령
        return "redirect:/new-url";
    }

    @GetMapping("/new-url")
    public String newPage() {
        return "new-page-view"; // 실제 보여줄 HTML 파일 이름
    }
}
```

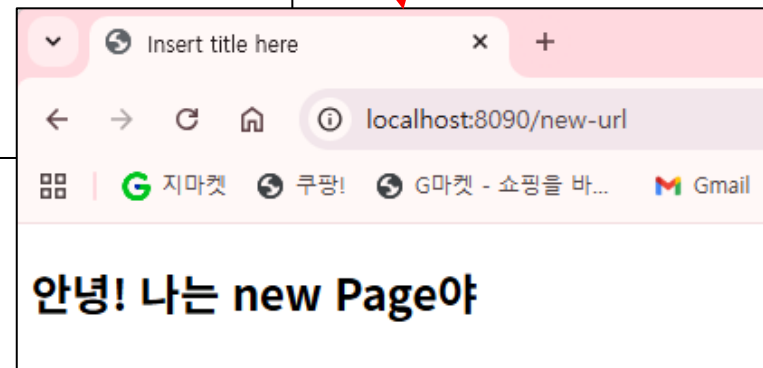


spring

➤ 리다이렉트(redirect) 사용하기

- sample_redirect 프로젝트 생성 -> src/main/resource -> new-page-view.html 생성
- `return "redirect:/new-url"` 은 HTML 파일을 찾는 이름이 아니라, 브라우저에게 보내는 명령어이다. 야, 브라우저야! 너 지금 즉시 **/new-url이라는 주소로 다시 접속해!**라고 시키는 것이다..
- 고로, new-url.html 파일을 필요하지 않다. 여기서는 new-page-view.html만 생성하면 된다.

```
<!DOCTYPE html>
<html lang="kor" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h2>안녕! 나는 new Page야</h2>
</body>
</html>
```





spring

➤ 리다이렉트(redirect) – RedirectAttributes 란

- redirect 할 때, 딱 한 번만 사용할 데이터를 전달하기 위한 객체이다.
- redirect는 데이터를 못 들고 가니까, 잠깐 맡겨두는 택배함과 같은 역할을 한다.
- redirect 이후 **딱 한 번만 사용 가능하고**, 새로고침하면 사라진다.
- Controller → FlashMap(임시저장) → Redirect → 다음 요청에서 Model로 복원 → 사용 후 자동 삭제



spring

➤ 리다이렉트(redirect) - RedirectAttributes 사용하기

- sample_redirect 프로젝트 생성 -> src/main/java -> MyController.java 생성
- 스프링 부트 컨트롤러에서 리다이렉트를 구현하는 방법은 아주 간단하다. 반환하는 문자열 앞에 **redirect:** 접두사를 붙여 준다.

```
@Controller
public class MyController {

    @GetMapping("/old-url")
    public String oldPage(RedirectAttributes re) {
        // 리다이렉트 시점에 딱 한 번만 사용할 데이터를 보관함
        re.addFlashAttribute("msg", "이전 주소에서 자동으로 이동되었습니다!");

        return "redirect:/new-url";
    }

    @GetMapping("/new-url")
    public String newPage() {
        return "new-page-view"; // 실제 보여줄 HTML 파일 이름
    }
}
```

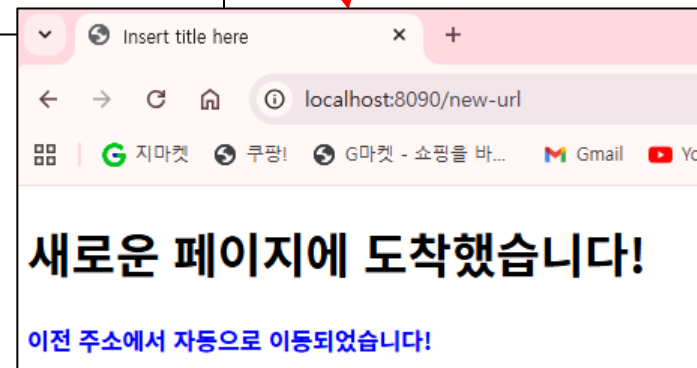



spring

➤ 리다이렉트(redirect) - RedirectAttributes 사용하기

- sample_redirect 프로젝트 생성 -> src/main/resource -> new-page-view.html 생성
- RedirectAttributes는 /old-url을 거쳐서 리다이렉트가 발생할 때만 데이터를 실어서 보낸다.
- 만약 주소창에 직접 /new-url을 치고 들어가면, 배달부(RedirectAttributes)를 거치지 않았기 때문에 msg 데이터가 전달되지 않는다.

```
1 <!DOCTYPE html>
2 <html lang="kor" xmlns:th="http://www.thymeleaf.org">
3 <head>
4 <meta charset="UTF-8">
5 <title>Insert title here</title>
6 </head>
7 <body>
8 <h1>새로운 페이지에 도착했습니다!</h1>
9
10 <p th:if="${msg}" th:text="${msg}" style="color: blue; font-weight: bold;"></p>
11 </body>
12 </html>
```





➤ 여기서 잠깐 !

- sample_redirect 프로젝트 -> src/main/java -> QuizController.java 생성한다.
- sample_redirect 프로젝트 -> src/main/resource -> main-view.html , quiz-view.html을 생성한다.

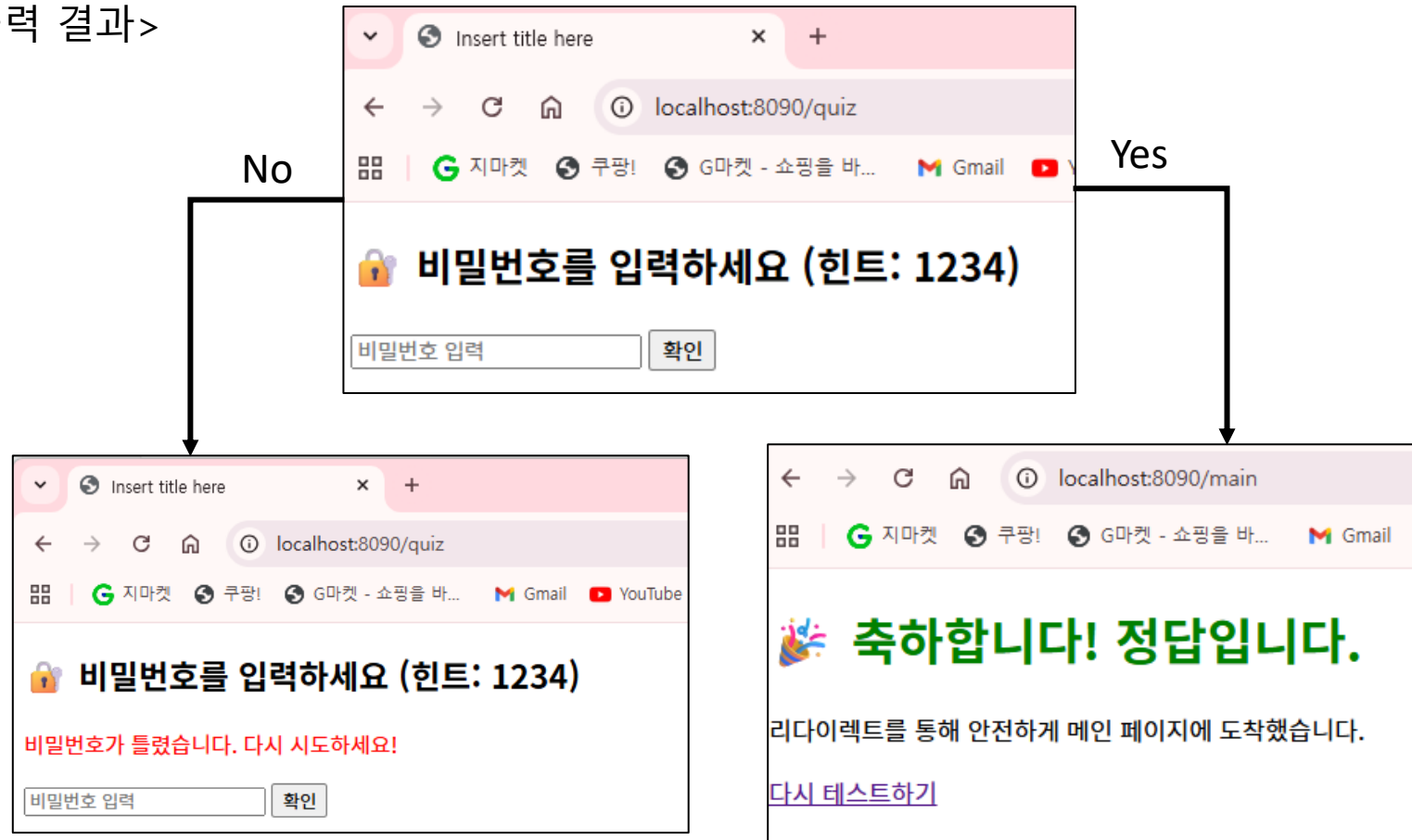
조건

- ① 비밀번호 확인 시스템
- ② 사용자가 비밀번호를 입력했을 때, 맞으면 메인 페이지로, 틀리면 다시 입력 페이지로 이동하는 리다이렉트 프로젝트를 개발 하시오.
- ③ @RequestParam("pass") String pass 을 이용해 개발하시오.
- ④ thymeleaf의 [th:if="\${errorMsg}" th:text="\${errorMsg}"] th:if를 이용해 개발한다.
- ⑤ 다시테스트하기 링크를 클릭하면 다시 비밀번호 입력하는 quiz화면으로 이동하도록 개발한다.
- ⑥ 다음 페이지의 <출력 결과>와 같이 출력되도록 개발하시오.



spring

<출력 결과>



thymeleaf





spring

➤ thymeleaf 란 ?

- 타임리프는 스프링 부트에서 공식적으로 권장하는 **템플릿 엔진**이다.
- HTML 파일 최상단 <html> 태그에 아래 주소를 넣어야 타임리프 문법을 사용할 수 있다.

```
<html lang="kor" xmlns:th="http://www.thymeleaf.org">
```

- 위의 <html>태그에 xmlns 부분을 넣어야 **th:**로 시작하는 속성들을 인식한다.

➤ thymeleaf 핵심 문법

문법	설명	사용 예시
<code>\${...}</code>	데이터 이름 불러오기	<code>\${msg}</code>
<code>th:text</code>	태그 안에 내용 쓰기	<code></code>
<code>th:if</code>	조건이 참일 때만 표시	<code><p th:if="\${msg}"></code>
<code>@{...}</code>	링크/URL 주소 적기	<code><a th:href="@{/main}"></code>



spring

➤ Thymeleaf 핵심문법 7

① th:text (텍스트 출력)

서버에서 보낸 데이터를 태그 안에 글자로 찍어준다.

```
<p th:text="${msg}">기본 글자</p>
```

② \${...} (변수 식)

변수

컨트롤러의 Model이나 RedirectAttributes에 담긴 데이터를 꺼낼 때 사용한다.

예: model.addAttribute("name", "홍길동") -> \${name}으로 꺼냄.

③ th:if (조건문) < T/F & null

특정 조건을 만족할 때만 해당 태그를 화면에 보여준다. (에러 메시지 띄울 때 필수!)

```
<p th:if="${errorMsg}" th:text="${errorMsg}" style="color: red;"></p>
```

th:if="\${errorMsg}" 의 의미는 \${errorMsg} 가 null이 아니고, 빈 문자열이 아니고, false가 아니면 <p> 태그 자체가 화면에 렌더링 됨, 만약 false나 null이면 <p>태그 자체가 HTML에 삭제되며 브라우저에 아무것도 생기지 않는다.



spring

➤ Thymeleaf 핵심문법 5

③ 다중 th:if (조건문)

```
<p th:if="${score} >= 90">A 학점</p>  
<p th:if="${score} >= 80 and score < 90">B 학점</p>  
<p th:if="${score} < 80">C 학점</p>
```

③ th:switch / th:case (다중 조건일 때 추천)

```
<div th:switch="${grade}">  
  <p th:case="A">우수</p>  
  <p th:case="B">보통</p>  
  <p th:case="C">미흡</p>  
  <p th:case="*">기타</p>  
</div>
```

③ 조건식 안에서 삼항 연산자도 가능

```
<span th:text="${isLogin ? '로그인됨' : '로그인 필요'}"></span>
```



spring

➤ Thymeleaf 핵심문법 5

④ **th:action** (폼 전송)

<form> 태그에서 데이터를 어디로 보낼지 주소를 지정한다.

```
<form th:action="@{/check-quiz}" method="post">
```

@{...}는 타임리프에서 주소(URL)를 나타내는 기호이다.

⑤ **th:each** (반복문)

리스트 데이터를 순회하며 반복해서 태그를 생성한다. (게시판 목록 등에 사용)

```
<tr th:each="post : ${postList}">
  <td th:text="${post.title}">제목 </td>
</tr>
```

⑥ **th:each** : 인덱스까지 쓰고 싶으면 (확장 for + index)

```
<tr th:each="post, stat : ${postList}">
  <td th:text="${stat.index}"></td>
  <td th:text="${post.title}"></td>
</tr>
```




spring

➤ Thymeleaf 핵심문법 5

⑥ th:each : stat

```
<tr th:each="post, stat : ${postList}">  
  <td th:text="${stat.index}"></td>  
  <td th:text="${post.title}"></td>  
</tr>
```

stat 값	의미
stat.index	0부터 시작하는 index
stat.count	1부터 시작
stat.size	전체 개수
stat.first	첫 번째면 true
stat.last	마지막이면 true
stat.even	짝수
stat.odd	홀수





spring

➤ Thymeleaf 핵심문법 5

⑦ **th:each + th:if**

th:each와 th:if를 같은 태그에 쓰면 each가 먼저 돌고, 그 다음 if가 필터처럼 걸러진다.

```
<tr th:each="post : ${postList}"  
    th:if="${post.notice}">  
    <td th:text="${post.title}"> </td>  
</tr>
```



```
for (Post post : postList) {  
    if (post.isNotice()) {  
        // 출력  
    }  
}
```

복습문제





➤ 여기서 잠깐 !

- sample_redirect 프로젝트 -> src/main/java -> AddressController.java 생성한다.
- sample_redirect 프로젝트 -> src/main/java -> AddressDTO.java 생성한다.
- sample_redirect 프로젝트 -> src/main/resource -> address-list.html을 생성한다.

조건

- ① 주소록 시스템
- ② 이름, 도시를 입력받아 주소록을 출력하는 프로그램을 개발하시오.
- ③ DTO, Redirect, thymeleaf, List를 이용해 개발하시오.



spring

- sample_redirect 프로젝트 -> src/main/java -> AddressDTO.java

```
public class AddressDTO {  
    private String name;  
    private String city;  
  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getCity() {  
        return city;  
    }  
    public void setCity(String city) {  
        this.city = city;  
    }  
}
```



spring

- sample_redirect 프로젝트 -> src/main/java -> AddressController.java

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class AddressController {
    // 메모리에 주소 데이터를 담은 리스트 (서버가 켜져 있는 동안 유지)
    private List<AddressDTO> addressList = new ArrayList<>();

    // 1. 주소록 목록 화면
    @GetMapping("/addresses")
    public String listPage(Model model) {
        model.addAttribute("list", addressList); // 리스트 전달
        return "address-list";
    }

    // 2. 주소 등록 처리
    @PostMapping("/add-address")
    public String addAddress(AddressDTO dto) { // @RequestParam 대신 DTO로 바로 받기
        addressList.add(dto);
        return "redirect:/addresses";
    }
}
```



spring

- sample_redirect 프로젝트 -> src/main/resources -> address-list.html

```
<h2>주소록 등록</h2>
<form action="@{/add-address}" method="post">
  이름: <input type="text" name="name" required><br>
  도시: <input type="text" name="city" required><br>
  <button type="submit">주소 추가하기</button>
</form>
<hr>
<h2>주소록 목록</h2>
<table>
  <thead>
    <tr>
      <th>이름</th>
      <th>도시</th>
    </tr>
  </thead>
  <tbody>
    <tr th:each="addr : ${list}">
      <td th:text="${addr.name}">이름</td>
      <td th:text="${addr.city}">도시</td>
    </tr>
    <tr th:if="${#lists.isEmpty(list)}">
      <td colspan="2">등록된 주소가 없습니다.</td>
    </tr>
  </tbody>
</table>
```



spring

<출력 결과>

Insert title here

localhost:8090/addresses

지마켓 쿠팡! G마켓 - 쇼핑을 바...

주소록 등록

이름:

도시:

주소 추가하기

주소록 목록

이름	도시
등록된 주소가 없습니다.	



Insert title here

localhost:8090/addresses

지마켓 쿠팡! G마켓 - 쇼핑을 바... Gmail

주소록 등록

이름:

도시:

주소 추가하기

주소록 목록

이름	도시
홍길동	서울시 동작구
개나리	경기도 부천시
진달래	인천 남동구



spring

➤ 주소는 이름표(Mapping)일 뿐이다

- 스프링 부트 컨트롤러에 적는 주소는 서버가 "이런 이름의 요청이 들어오면 이 메서드를 실행해!" 라고 정해둔 **이름표**이다.
- **실제 물리적 위치**: src/main/java/.../MyController.java (자바 파일 위치)
- **가상의 논리 주소**: http://localhost:8080/member/board
- 사용자가 브라우저에 /member/board라고 쳤다고 해서 서버 컴퓨터의 member 폴더 안의 board 파일을 찾아가는 게 아니다. 스프링이 "**어? 이 주소 이름표 달고 있는 컨트롤러 메서드 어디 있지?**" 하고 찾아서 실행하는 방식이다.

➤ 리턴하는 HTML 파일

- 주소(@GetMapping)는 폴더와 상관없지만, 메서드 안에서 **return** 하는 **문자열**은 실제 폴더 경로를 의미할 수 있다.
- **return "member/..."**: templates 폴더 아래에 실제로 **member**라는 폴더가 있어야 함.



spring

➤ 이름표(Mapping) 한 줄 정리

구분	@GetMapping("/a/b")	return "a/b"
정체	URL 주소 (브라우저 주소창)	파일 경로 (내 컴퓨터 안의 위치)
성격	가상의 이름표 (약속)	실제 폴더와 파일 위치
폴더 필요성	필요 없음!	반드시 있어야 함!